Jeevana Raksha Pharmacy

MySQL Database Project Report

Prepared by
Matcha Sai Satish Chandra Kumar
Course Code: JFS-ONR-005
Technology: MySQL

A relational database case study for pharmacy inventory, customers, suppliers, and sales orders.

1. Case Study Overview

Store name: Jeevana Raksha Pharmacy

Location: India

Project objective: To design a relational database that organizes pharmacy customers, suppliers, medicine inventory, orders, and order line items, and to use SQL queries to retrieve useful operational and sales information.

Business Entities

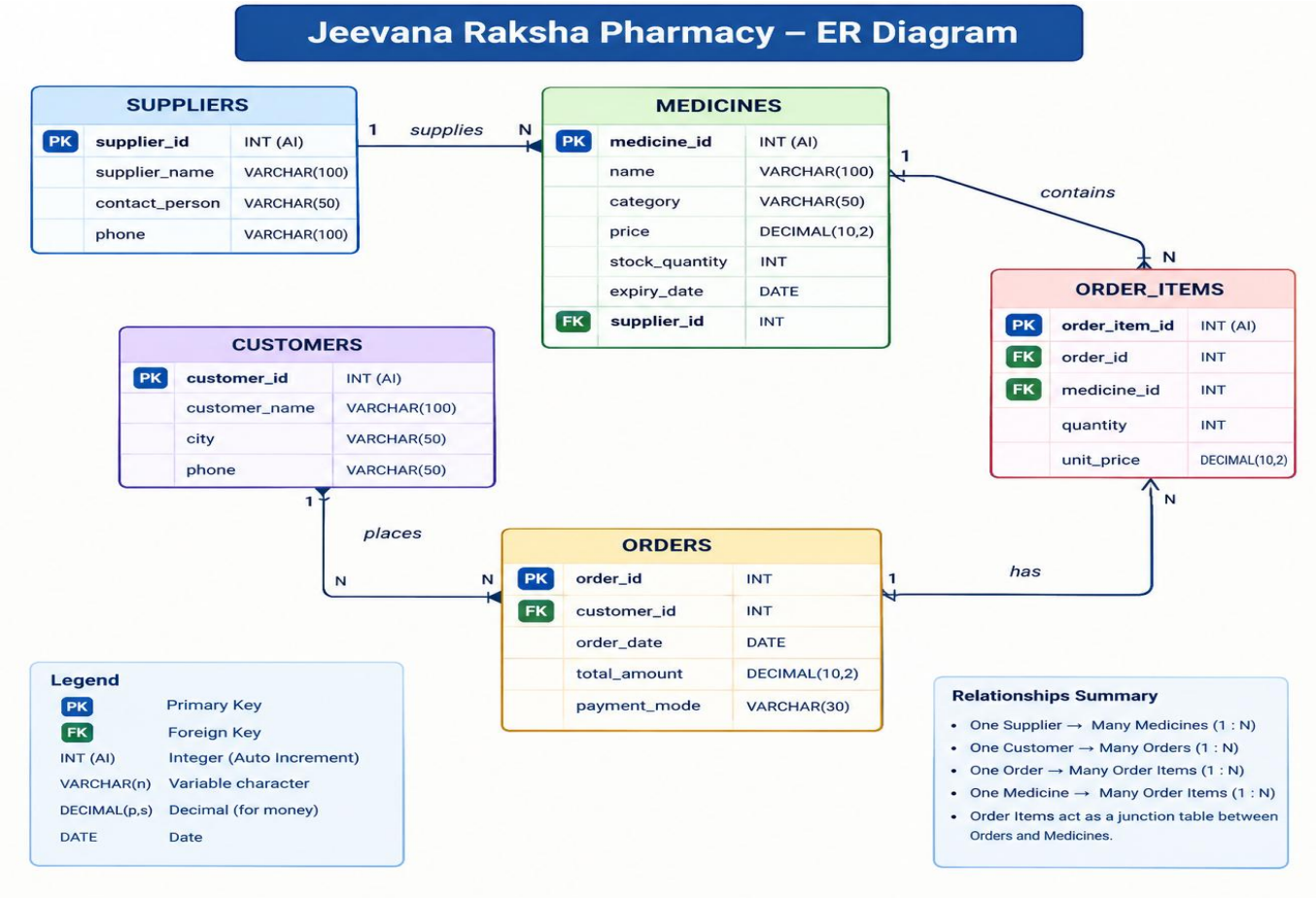
- **Customers:** Stores customer names, cities, and phone numbers.
- **Suppliers:** Stores supplier names, contact persons, and phone numbers.
- **Medicines:** Stores medicine names, categories, prices, stock quantities, expiry dates, and supplier references.
- **Orders:** Stores customer order dates, total amounts, and payment modes.
- **Order Items:** Stores the medicines included in each order, their quantities, and unit prices.

2. Database Design

The database contains five related tables. Primary keys uniquely identify records, while foreign keys connect related records and help maintain referential integrity.

Table	Primary Key	Foreign Key(s)	Purpose
customers	customer_id	—	Customer contact details
suppliers	supplier_id	—	Supplier contact details
medicines	medicine_id	supplier_id → suppliers.supplier_id	Medicine inventory and supplier mapping
orders	order_id	customer_id → customers.customer_id	Order header and payment details
order_items	order_item_id	order_id → orders.order_id; medicine_id → medicines.medicine_id	Line items within each order

Entity Relationship Diagram (ERD)



Relationship summary:

- One supplier can be associated with many medicines (1:N).
- One customer can place many orders (1:N).
- One order can contain many order items (1:N).
- One medicine can appear in many order items (1:N).
- The order_items table connects orders and medicines.

3. SQL Exercises and Solutions

The following queries demonstrate basic retrieval, filtering, joins, aggregation, grouping, and inventory analysis.

1. Customer List

Requirement: Retrieve the name and city of all customers.

```
SELECT customer_name, city
FROM customers;
```

Explanation: Uses SELECT to choose the required columns and FROM to identify the source table.

2. Stock Check

Requirement: Find medicines in the Syrup or Injection categories.

```
SELECT *
FROM medicines
WHERE category IN ('Syrup', 'Injection');
```

Explanation: Uses WHERE to filter rows and IN to match either category.

3. High-Value Orders

Requirement: List orders with a total amount greater than ₹500.

```
SELECT *
FROM orders
WHERE total_amount > 500;
```

Explanation: Uses a comparison operator in WHERE to return orders above the specified amount.

4. Mumbai Residents

Requirement: Find the names and phone numbers of customers living in Mumbai.

```
SELECT customer_name, phone
FROM customers
WHERE city = 'Mumbai';
```

Explanation: Filters customers by city and returns only the requested columns.

5. Medicines by Supplier

Requirement: Count medicines supplied by Apollo Distributors.

```
SELECT COUNT(m.medicine_id) AS total_medicines
FROM medicines m
JOIN suppliers s
  ON m.supplier_id = s.supplier_id
WHERE s.supplier_name = 'Apollo Distributors';
```

Explanation: Uses INNER JOIN to connect medicines with suppliers, then COUNT to count matching medicine records.

6. Order Details

Requirement: Show the customer name, order date, and total amount for each order.

```
SELECT c.customer_name, o.order_date, o.total_amount
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id;
```

Explanation: Joins orders with customers using customer_id so each order can be displayed with its customer name.

7. Revenue by Payment Mode

Requirement: Calculate the total recorded order amount for each payment mode.

```
SELECT payment_mode, SUM(total_amount) AS total_money
FROM orders
GROUP BY payment_mode;
```

Explanation: SUM adds the amounts and GROUP BY produces one total for each payment mode.

8. Best-Selling Medicine

Requirement: Find the medicine with the greatest total quantity sold.

```
SELECT m.name, SUM(oi.quantity) AS total_quantity
FROM order_items oi
JOIN medicines m
  ON oi.medicine_id = m.medicine_id
GROUP BY m.medicine_id, m.name
ORDER BY total_quantity DESC
LIMIT 1;
```

Explanation: Joins order items to medicine names, totals quantities per medicine, sorts descending, and returns the top row. Remove LIMIT 1 to display the full ranking.

9. Customer Spending

Requirement: List customers whose total recorded order amount exceeds ₹1,000.

```
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.total_amount) > 1000;
```

Explanation: GROUP BY calculates a total per customer. HAVING filters groups based on the aggregate SUM.

10. Low-Stock Inventory

Requirement: List medicines with fewer than 50 units, along with supplier names.

```
SELECT m.name, m.stock_quantity, s.supplier_name
FROM medicines m
JOIN suppliers s
  ON m.supplier_id = s.supplier_id
WHERE m.stock_quantity < 50;
```

Explanation: Joins medicine and supplier data, then filters inventory below 50 units.

4. Key SQL Concepts Demonstrated

- SELECT and WHERE: retrieve selected columns and filter records.
- IN and comparison operators: match multiple values or apply numeric conditions.
- INNER JOIN: combine related rows from tables using key relationships.
- COUNT and SUM: calculate record counts and totals.
- GROUP BY: aggregate rows by a shared category or customer.
- HAVING: filter grouped results using aggregate conditions.
- ORDER BY and LIMIT: rank results and return the top record.

5. Conclusion

The Jeevana Raksha Pharmacy MySQL project models key pharmacy operations using five related tables. The SQL exercises demonstrate how relational data can be queried to review customers, supplier relationships, order values, payment-mode totals, medicine demand, customer spending, and low-stock items. The project provides practice with core SQL concepts and relational database design.